BUY THE DIP

AI Workshop Day 2 Project Report

A Claude-powered options trading copilot built with FastAPI, Next.js, Alpaca paper trading, and a human-in-the-loop approval model.

Workshop context:	Second day of an AI workshop; focus on turning an idea into a working demo.
Core idea:	An agent analyzes a ticker, proposes an options trade, and streams its process live.
Safety invariant:	The agent cannot execute trades; the user must approve each paper order.

Prepared for workshop judges and reviewers

`Repository: github.com/Gustavious-slalom/buy-the-dip`

Executive Summary

Buy the Dip is a two-day proof-of-concept trading platform created during an AI workshop. The project demonstrates how an LLM-powered agent can combine market data, portfolio context, news, options chains, and a human approval workflow to produce observable paper-trade proposals.

By Day 2, the project has moved beyond a prompt or mockup: it has a FastAPI backend, a Next.js dashboard, a streaming WebSocket agent trace, Alpaca paper-trading integration, SQLite persistence, replay support, and a new portfolio composition view for account health, positions, allocations, strategies, and equity history.

- The project is intentionally scoped for a hackathon-style demo: single-user, local-first, paper trading only, fixtures for offline fallback, and clear safety boundaries.
- The strongest demo moment is transparency: judges can watch the agent call tools, reason about market context, create a proposal, and wait for explicit human approval before execution.

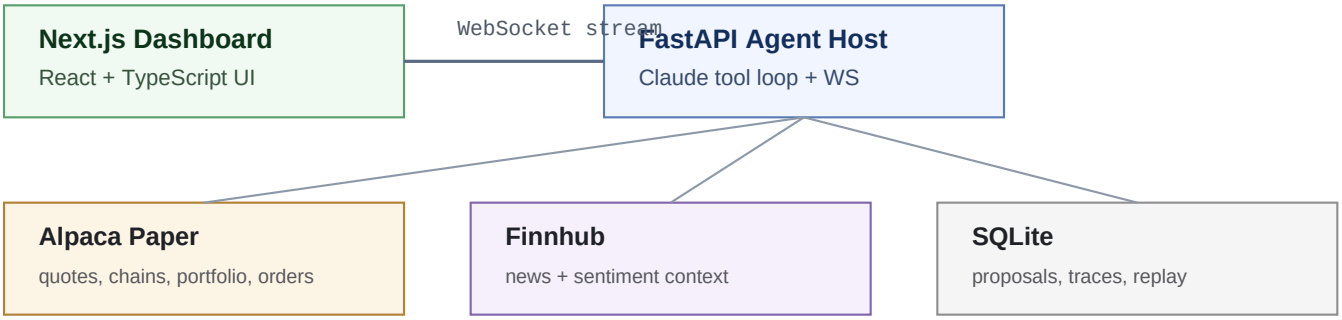
PROJECT GOALS

- Show a practical AI-agent workflow rather than a static chatbot.
- Use real trading primitives: quotes, options chains, Greeks, news, portfolio state, and order submission.
- Keep execution safe by separating proposal generation from paper-order submission.
- Create a polished dashboard that makes the agent observable and reviewable.
- Make the demo resilient with SQLite traces, replay, fixtures mode, and local deployment.

Technology Stack

Layer	What was used	Why it mattered
Backend	Python, FastAPI, SQLModel, SQLite	Fast async API, WebSockets, finance ecosystem, low...
Frontend	Next.js, React, TypeScript, Tail...	Fast dashboard iteration with typed components
Charts	Recharts	Price and portfolio visuals without heavy custom c...
Agent	Claude Sonnet + Haiku models	Tool-use loop, streaming analysis, lower-cost seco...
Market data	Alpaca paper trading	Quotes, options, account data, and safe paper exec...
News	Finnhub	Recent company news for sentiment/context

Architecture



The system is split into two runnable processes: a Next.js dashboard and a FastAPI backend. The browser starts an analysis session over WebSocket, receives streamed agent events, then calls REST endpoints to approve or reject pending proposals.

The backend hosts the Claude tool loop and service adapters. It calls Alpaca for market and account data, Finnhub for news, and SQLite for proposals, executions, traces, watchlists, portfolio history joins, and replayable agent sessions.

- WebSocket: /ws streams agent.status, agent.thinking, agent.tool_call, agent.tool_result, agent.proposal, agent.complete, and agent.error events.
- REST: /proposals, /proposals/approve, /proposals/reject, /bars/{symbol}, /portfolio/snapshot, and /portfolio/equity-curve.
- Startup safety: the backend asserts Alpaca is configured for paper trading before serving requests.

Backend Build

FASTAPI APPLICATION SHELL

The FastAPI app initializes settings, creates database tables, enables CORS for the local dashboard, and mounts WebSocket and HTTP routers. The backend is deliberately small and local-friendly so a workshop team can run it quickly.

AGENT LOOP

The agent loop uses Anthropic streaming messages with tool definitions and a system prompt. It streams text deltas to the UI as agent.thinking, dispatches tool calls asynchronously, emits tool results, and stops after a fixed 12-iteration cap to avoid runaway loops.

TOOLS EXPOSED TO CLAUDE

- get_quote(symbol)
- get_options_chain(symbol, expiry)
- get_greeks(contract_symbol)
- get_news(symbol, since_days)
- get_portfolio()
- get_positions()
- propose_trade(legs, rationale, risks, expiry)

Notably, execute_trade is not exposed as an agent tool. The agent can create a pending proposal, but execution belongs to the REST approval path controlled by the user interface.

Safety and Human Approval

The most important product decision is the safety invariant: the LLM does not submit orders. A proposal is stored as pending, displayed in the UI with rationale and risk, and only then can the user approve or reject it.

- Approval endpoint checks that the proposal exists and is still pending.
- Approval rejects trades above MAX_RISK_USD.
- Execution uses Alpaca paper trading and records an Execution row with order status and raw response.
- Rejection changes proposal status without touching Alpaca.
- Failures are surfaced as errors and persisted as failed execution attempts when relevant.

Frontend Build

The Next.js frontend is organized around a live terminal-style dashboard. The trading page uses a three-column layout: ticker input and portfolio summary on the left, proposal/charts/options chain in the center, and the agent trace on the right.

- TickerInput starts an agent session and supports replaying a stored trace.
- AgentTrace renders streamed thinking, tool calls, and tool results so the AI process is visible.
- ProposalCard presents legs, expiry, confidence, max risk, max reward, breakeven, rationale, risks, and Approve/Reject actions.
- PriceChart and OptionsChainTable turn tool results into market context for the proposed trade.
- StatusRail adds the workshop-ready terminal aesthetic and navigation between Trade and Portfolio.

Day 2 Portfolio Work

The second workshop day expanded the project from a trade-proposal demo into a broader trading platform surface. The new /portfolio route gives judges a snapshot of what the paper account contains and how agent-generated trades show up after approval.

Area	Day 2 addition
Route	/portfolio page linked from the top status rail
Backend	portfolio_service.py aggregates Alpaca account/positions with local proposals
API	/portfolio/snapshot and /portfolio/equity-curve?period=...
Components	Account summary, equity curve, allocation card, strategies list, positions table, ...
State refresh	Manual refresh and automatic portfolio invalidation after approve/reject
Offline support	Fixture-backed portfolio history for demo resilience

The portfolio view groups open option positions back to executed proposals where possible. This makes multi-leg strategies understandable to reviewers instead of presenting each option leg as an isolated line item.

Data Model and Persistence

SQLite is used because the workshop goal is a reliable PoC, not production infrastructure. SQLModel provides simple typed models while keeping setup lightweight.

Model	Purpose
Proposal	Pending/executed/rejected/failed trade ideas with legs, risk, reward, rationale, ...
Execution	Alpaca paper order submission result linked to a proposal
Trace	Every streamed agent event for replay and auditability
Watchlist	Seeded symbols such as AAPL, NVDA, and SPY

Trace persistence is especially valuable for a workshop demo: even if a live market-data API has a hiccup, the team can replay a previously captured agent session and still demonstrate the end-to-end UX.

AI Workshop Creation Timeline

Phase	What was created
Day 1 AM	Core architecture, FastAPI backend, Anthropic tool loop, Alpaca service adapters, ...
Day 1 PM	WebSocket streaming, Next.js dashboard, trace UI, proposal approval flow, charts/o...
Day 2 AM	Portfolio composition design and backend aggregation: account, positions, allocati...
Day 2 PM	Portfolio UI polish, status-rail navigation, invalidation after approve/reject, do...

AI assistance was used as an accelerator for planning, decomposition, implementation sequencing, and code generation. The project benefited from pairing AI-generated structure with human decisions about scope, safety, and what would make the demo understandable.

Demo Script for Judges

1. Start backend and frontend locally. If keys or network access are unreliable, enable fixture mode.
2. Open the dashboard and enter a ticker such as NVDA.
3. Narrate the live agent trace: quote, news, options chain, portfolio, positions, proposal.
4. Show the proposal card: legs, expiry, confidence, max risk, max reward, breakeven, rationale, and risks.
5. Approve the paper trade and point out that approval happens outside the LLM tool loop.
6. Navigate to Portfolio and show account summary, equity curve, allocation, grouped strategies, positions, and history.

7. Use Replay Last if a live API hiccup happens during presentation.

Validation and Current Status

The repository includes backend pytest coverage for models, proposal services, Alpaca service behavior, news service behavior, agent loop/tooling, portfolio API, and portfolio aggregation helpers. The frontend includes TypeScript and production build scripts.

- Backend tests exercise allocation math, OCC option parsing, position normalization, strategy grouping, partial failure handling, and equity-curve period validation.
- Frontend checks include pnpm tsc --noEmit and pnpm build as the main validation path.
- The README documents setup, environment variables, offline fixture mode, seed data, tests, and the five-minute demo flow.

Current project status: a functional workshop PoC with the key safety, observability, and portfolio-review pieces in place. The remaining work is production hardening, richer broker/error handling, authentication if needed, and deeper trading analytics beyond the PoC scope.

Recommended Next Steps

- Record a fallback demo video using fixture mode and one successful replay trace.
- Tighten visual polish around empty/error states before presenting.
- Add more realistic strategy labels and risk summaries for spreads.
- Document API-key setup clearly for anyone running the project after the workshop.
- Keep all execution paper-only unless the project later receives a full compliance and risk review.

Closing

Buy the Dip is a strong Day 2 workshop artifact because it shows more than generated code. It shows a complete AI-assisted product loop: plan the architecture, build the agent host, stream the AI process, require human approval, persist the audit trail, and present the resulting portfolio in a polished interface.

The project demonstrates how modern AI tools can help a small team move from concept to working demo quickly while still making deliberate engineering choices around safety, observability, and reliability.